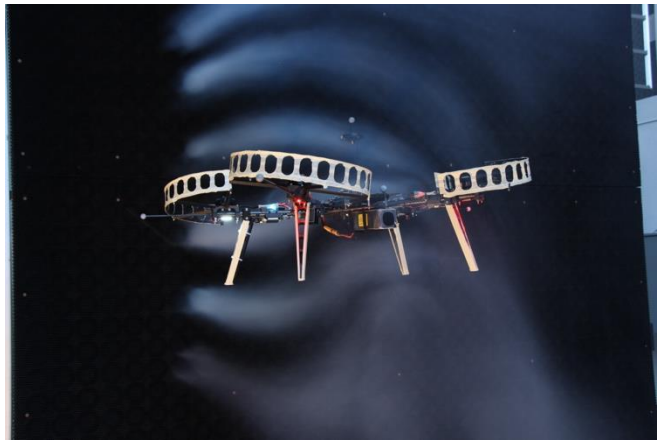
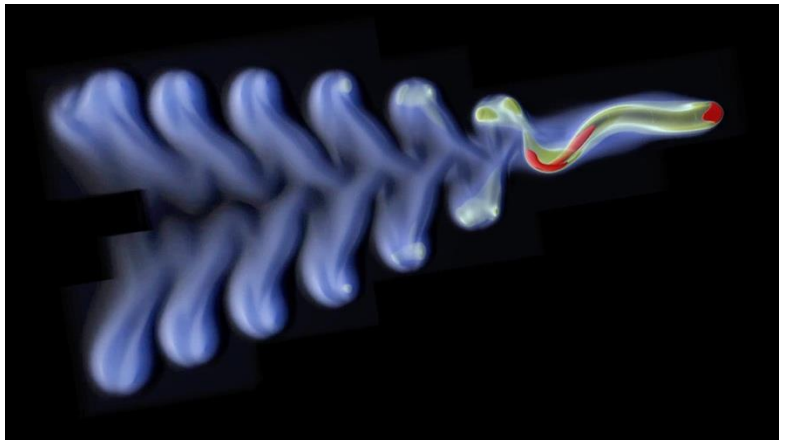
16-831: Introduction to Robot Learning, Fall 2024

Optimal Control

$$\dot{x} = \frac{dx}{dt} = f(x, u) \qquad x_{t+1} = f(x_t, u_t)$$



Aerodynamics in wind, *Neural-Fly*



Fluid dynamics, MIT van Rees Lab

Offroad vehicle dynamics, UW Racer team

Instructor: Guanya Shi

LeCAR
Learning and Control for Agile Robotics Lab

Carnegie Mellon University
The Robotics Institute

Recording Reminder

- This session will be audio recorded for educational use by other students in this course.

Project Proposal

- ❑ DDL Oct 11, this Friday (no extension)
- ❑ No grade but mandatory for feedback

Course Structure

Latest schedule: https://16-831.github.io/fall24/lectures

- ❑ Topic group 1: Course introduction: What is robot learning and why?
- ❑ Topic group 2: Machine/Deep learning refresher
- ❑ Topic group 3: Imitation learning
- ❑ Topic group 4: Model-free reinforcement learning
- ❑ Topic group 5: Model-based reinforcement learning
- ❑ Topic group 6: Bandit, preference-based learning, exploration
- ❑ Topic group 7: Reinforcement learning from offline data (one guest lecture)
- ❑ Topic group 8: Specialized topics (one guest lecture)
- ❑ Project presentation

Outline of the MBRL Topic Group

- Part I: control theory basics
  - Stability and Lyapunov methods for stability
  - Uncertainty and robust control
  - Adaptive control
  - Linear systems and continuous-time LQR
  - Linearization of nonlinear systems
- Part 2: how can we make decisions if we *know* the dynamics?
  - Trajectory optimization (TO) and model predictive control (MPC)
  - Discrete time LQR
  - Sequential quadratic/convex programming (SQP/SCP)
  - iLQR/DDP
  - Sampling-based MPC
- Part 3: learn *unknown* dynamics and use it for control
  - Model learning
  - Policy optimization with the learned model
  - Deep MBRL: Dreamer, MBRL+MFRL, etc

Recap: Control Theory Basics

- Given the dynamics model $\dot{x} = f(x, u)$ or $x_{t+1} = f(x_t, u_t)$
- Design a *feedback* policy $u = \pi(x)$ achieving some goals:
  - Stabilization/regularization to some reference point x_d
  - Track a desired trajectory $x_d(t)$
  - **Minimize some cost functions** – it is where RL started!
  - Adaptive to uncertainty
  - Robust to disturbance
- Control theory: rigorously define and analyze these "goals"

Model-based RL and Optimal Control

- MBRL and optimal control are essentially both solving:
  - c_t is cost function
  - X_t is the state constraint (e.g., collision avoidance)
  - U_t is the control constraint (i.e., actuation limit)
  - T is the horizon
    - Can be fixed and finite, infinite, or a decision variable
    - Become a time-optimal control problem when T is a decision variable

$$\min_{x,u} \sum_{t=1}^{T} c_t(x_t, u_t)$$

$$\text{s.t.} \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ x_t \in X_t \\ u_t \in U_t \end{cases}$$

Trajectory/Policy Optimization

- Open-loop trajectory optimization (planning):
  - Solve a trajectory $x_{1:T}, u_{1:T}$ for low-level feedback control
- Closed-loop policy optimization:
  - Find a policy $u = \pi(x)$ to minimize the cost
- MBRL is basically optimal control with an unknown model
- Today: $f_{1:T}$ is *known*
- Next lecture: $f_{1:T}$ is *unknown*

$$\min_{x,u} \sum_{t=1}^{T} c_t(x_t, u_t)$$

$$\text{s.t.} \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ x_t \in X_t \\ u_t \in U_t \end{cases}$$

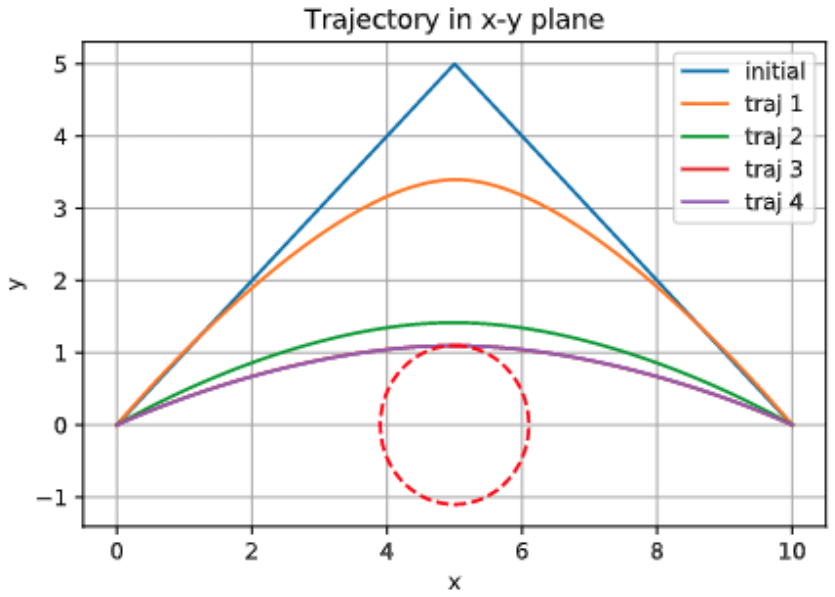
TO Example: Double Integrator Dynamics

$$\min_{x,u} \sum_{t=1}^{T} \|u_t\|_2^2$$

$$\text{s.t.} \begin{cases} x_{t+1} = f(x_t, u_t) \\ x_0 = [0,0], x_T = [10,0], x_t \notin \text{red circle} \\ u_t \in U \end{cases}$$

$$\dot{\mathbf{x}} = \underbrace{\begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}}_{A} \mathbf{x} + \underbrace{\begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}}_{B} \underbrace{\begin{bmatrix} u_1 \\ u_2 \end{bmatrix}}_{\mathbf{u}}$$

- Double integrator dynamics
- Minimum control energy
- Nonconvex: where is the nonconvexity from?
- Solved by SCP (sequential convex programming)



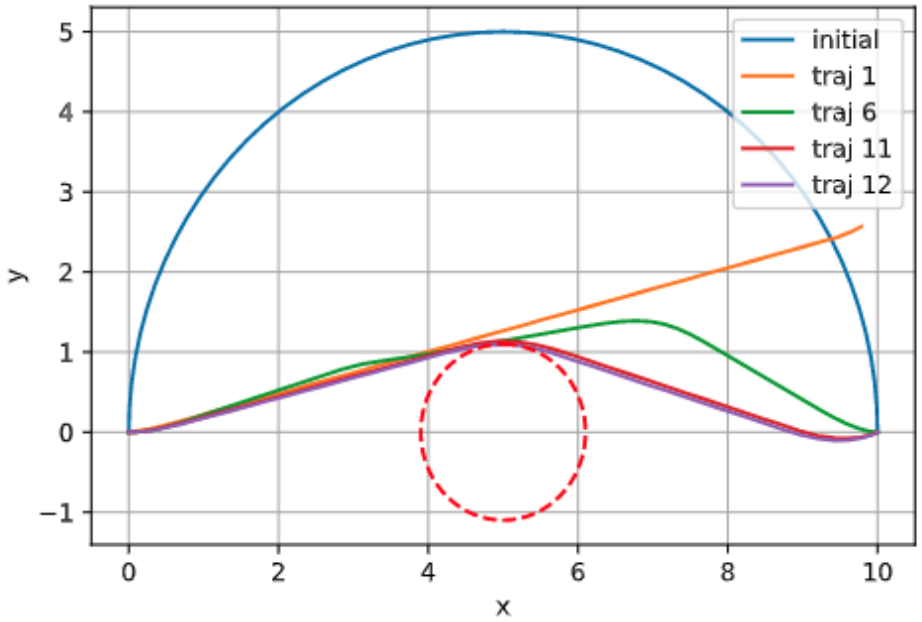
TO Example: Unicycle

$$\min_{x,u} \sum_{t=1}^{T} \|u_t\|_2^2$$

$$\text{s.t.} \begin{cases} x_{t+1} = f(x_t, u_t) \\ x_0 = [0,0], x_T = [10,0], x_t \notin \text{red circle} \\ u_t \in U \end{cases}$$

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ u_1 \cos\theta \\ u_1 \sin\theta \\ u_2 \end{bmatrix} = f(\mathbf{x}, \mathbf{u}).$$

- ❑ Unicycle dynamics
- ❑ Other things same
- ❑ “More” nonconvex: where is nonconvexity from?
- ❑ Solved by SCP (sequential convex programming)

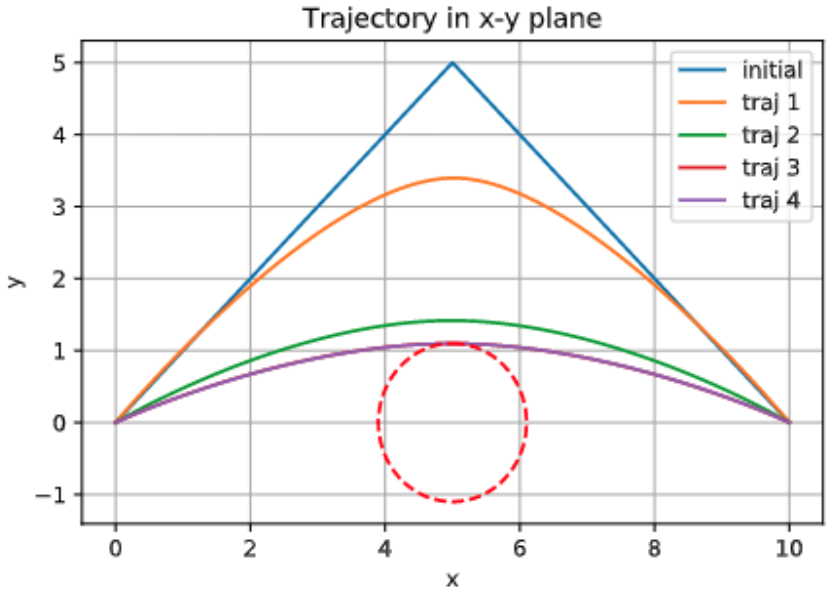


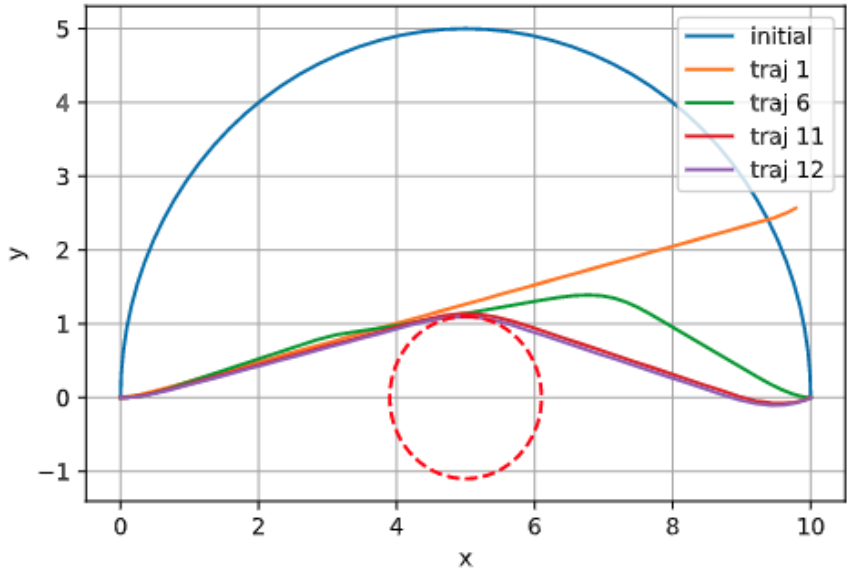
Dynamics Matters!

- Compare two examples
  - It is why dynamics matters a lot!
  - Question: what happens if we ignore dynamics?

$$\dot{\mathbf{x}} = \underbrace{\begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}}_{A} \mathbf{x} + \underbrace{\begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}}_{B} \underbrace{\begin{bmatrix} u_1 \\ u_2 \end{bmatrix}}_{\mathbf{u}}$$

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ u_1 \cos\theta \\ u_1 \sin\theta \\ u_2 \end{bmatrix} = f(\mathbf{x}, \mathbf{u}).$$



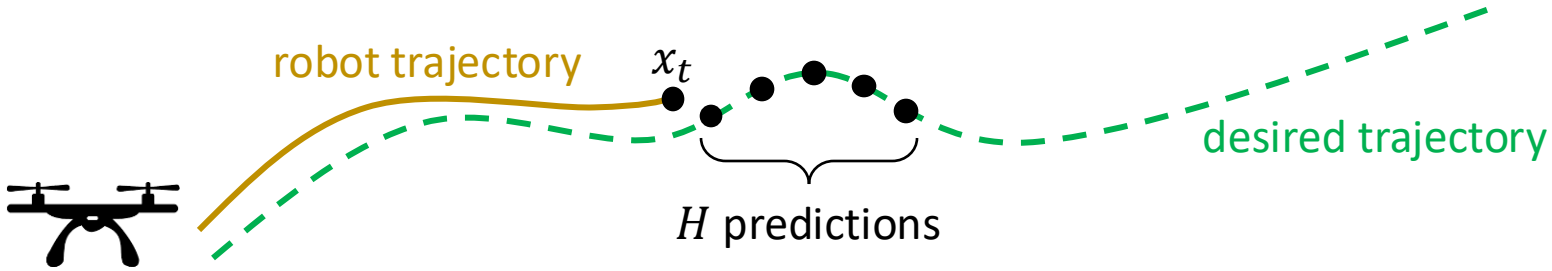


Model Predictive Control (MPC)

- MPC: solve a H-step TO problem ($H \ll T$) at *every time step*
- At time step t (robot is at x_t), solve the following optimization problem:
  - Solution is $u^*_{t|t}, u^*_{t+1|t}, \cdots, u^*_{t+H-1|t}$
  - **Key of MPC**: only execute $u^*_{t|t}$, and resolve the problem at the next time step

$$\min_{u} \sum_{h=0}^{H-1} c_{t+h}(x_{t+h}, u_{t+h}) + V_t(x_{t+H})$$

$$\text{s.t.} \begin{cases} x_{t+h+1} = f_t(x_{t+h}, u_{t+h}), \forall h \in [0, H-1] \\ x_{t+h} \in X_{t+h}, \forall h \in [1, H] \\ u_{t+h} \in U_{t+h}, \forall h \in [0, H-1] \end{cases}$$



Model Predictive Control (MPC)

- Forward-moving nature of the prediction horizon
  - Also called receding horizon control
- Implicitly closed-loop
  - Because $u^*_{t|t}$ depends on x_t
- The cost-to-go function (a.k.a. terminal cost or value function) V_t is important!
  - Better V_t allows shorter horizon (smaller H)
  - V_t is from heuristics, offline data/learning (e.g., value function), or online estimation

$$\min_{u} \sum_{h=0}^{H-1} c_{t+h}(x_{t+h}, u_{t+h}) + V_t(x_{t+H})$$

$$\text{s.t.} \begin{cases} x_{t+h+1} = f_t(x_{t+h}, u_{t+h}), \forall h \in [0, H-1] \\ x_{t+h} \in X_{t+h}, \forall h \in [1, H] \\ u_{t+h} \in U_{t+h}, \forall h \in [0, H-1] \end{cases}$$

How to Solve a TO/MPC Problem?

- When convex, relatively straightforward
  - In particular, quadratic programming (QP)
  - QP w/o state and action constraints: LQR
- When nonconvex:
  - Local optimization method: sequential quadratic/convex programming (SQP/SCP)
  - Dynamic programming (DP) + local optimization: iLQR/DDP
  - Sampling-based methods: MPPI, CEM, CMA-ES

$$\min_{x,u} \sum_{t=1}^{T} c_t(x_t, u_t)$$

$$\text{s.t.} \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ x_t \in X_t \\ u_t \in U_t \end{cases}$$

QP-based Trajectory Optimization

- When the TO problem is QP, relatively straightforward to solve
- There are many efficient solvers (e.g., OSQP, GUROBI, etc)
- In general, complexity increases *cubically* as T or $\dim\left(\begin{bmatrix} x_t \\ u_t \end{bmatrix}\right)$ increases
- QP is the basis of many (nonconvex) TO and MPC algorithms

$$\min_{x,u} \sum_{t=1}^{T} \frac{1}{2} \begin{bmatrix} x_t \\ u_t \end{bmatrix}^{\top} C_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^{\top} l_t$$

$$\text{s.t.} \begin{cases} x_{t+1} = F_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + g_t, \forall t \in [1, T-1] \\ d_t^{\min} \leq D_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} \leq d_t^{\max}, \forall t \in [1, T] \end{cases}$$

Linear Quadratic Regulator (LQR)

- Linear (affine) dynamics
- Quadratic cost
- Compared to the previous QP formulation: no state or action constraint!
  - It is why we can do dynamic programming with an analytic solution

$$\min_{x,u} \sum_{t=1}^{T} \frac{1}{2} \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top C_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top l_t$$

$$\text{s.t.} \quad x_{t+1} = F_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + g_t, \forall t \in [1, T-1]$$

- Notations:

$$C_t = \begin{bmatrix} Q_t & N_t \\ N_t^\top & R_t \end{bmatrix} \in \mathbb{R}^{(n+m)\times(n+m)} \succ 0$$

$$l_t = \begin{bmatrix} l_{x,t} \\ l_{u,t} \end{bmatrix} \in \mathbb{R}^{n+m}$$

$$F_t = \begin{bmatrix} A_t & B_t \end{bmatrix} \in \mathbb{R}^{n\times(n+m)}$$

Warmup: Simple Case (Standard LQR)

- No cross term ($N_t = 0$)
- No bias ($l_t, g_t = 0$)

$$\min_{x,u} \sum_{t=1}^{T} \frac{1}{2} x_t^\top Q_t x_t + \frac{1}{2} u_t^\top R_t u_t$$

$$\text{s.t.} \quad x_{t+1} = A_t x_t + B_t u_t, \forall t \in [1, T-1]$$

- Propose a quadratic optimal cost-to-go (value) function: $V_t(x_t) = \frac{1}{2} x_t^\top P_t x_t$
- Step 1: Verify it is true at time step T (last time step): Yes! $P_T = Q_T$
- Step 2: Assume it is true at time step $t+1$ (P_{t+1}), and do dynamic programming!
  - Bellman recursion:

$$V_t(x_t) = \min_u \underbrace{\frac{1}{2} x_t^\top Q_t x_t + \frac{1}{2} u^\top R_t u}_{\text{current step cost}} + \underbrace{V_{t+1}(A_t x_t + B_t u)}_{\text{cost-to-go}}$$

Warmup: Simple Case

$$\min_{x,u} \sum_{t=1}^{T} \frac{1}{2} x_t^\top Q_t x_t + \frac{1}{2} u_t^\top R_t u_t$$

$$\text{s.t.} \quad x_{t+1} = A_t x_t + B_t u_t, \forall t \in [1, T-1]$$

$$V_t(x_t) = \min_u \frac{1}{2} x_t^\top Q_t x_t + \frac{1}{2} u^\top R_t u + V_{t+1}(A_t x_t + B_t u)$$

$$= \min_u \frac{1}{2} x_t^\top (Q_t + A_t^\top P_{t+1} A_t) x_t + \frac{1}{2} u^\top (R_t + B_t^\top P_{t+1} B_t) u + u^\top B_t^\top P_{t+1} A_t x_t$$

$$u^* = -\underbrace{(R_t + B_t^\top P_{t+1} B_t)^{-1} B_t^\top P_{t+1} A_t}_{K_t} x_t$$

$$V_t(x_t) = \frac{1}{2} x_t^\top [\underbrace{Q_t + A_t^\top P_{t+1} A_t - A_t^\top P_{t+1} B_t (R_t + B_t^\top P_{t+1} B_t)^{-1} B_t^\top P_{t+1} A_t}_{P_t}] x_t$$

Warmup: Simple Case

$$\min_{x,u} \sum_{t=1}^{T} \frac{1}{2} x_t^\top Q_t x_t + \frac{1}{2} u_t^\top R_t u_t$$

$$\text{s.t.} \quad x_{t+1} = A_t x_t + B_t u_t, \forall t \in [1, T-1]$$

- Algorithm:
  - $P_T = Q_T$
  - Backward pass: $P_t = Q_t + A_t^\top P_{t+1} A_t - A_t^\top P_{t+1} B_t (R_t + B_t^\top P_{t+1} B_t)^{-1} B_t^\top P_{t+1} A_t$
  - Compute gain: $K_t = (R_t + B_t^\top P_{t+1} B_t)^{-1} B_t^\top P_{t+1} A_t$
  - Compute action: $u_t = -K_t x_t$
- Remarks:
  - Optimal total cost: $V_1(x_1) = \frac{1}{2} x_1^\top P_1 x_1$
  - LQR returns a *linear feedback controller*
  - Not only get a sequence of actions, also a sequence of feedback gains!

Warmup: Simple Case

$$\min_{x,u} \sum_{t=1}^{T} \frac{1}{2} x_t^\top Q_t x_t + \frac{1}{2} u_t^\top R_t u_t$$

$$\text{s.t.} \quad x_{t+1} = A_t x_t + B_t u_t, \forall t \in [1, T-1]$$

- For time-invariant system (A, B, Q, R are fixed):
  - $P_T = Q$
  - Backward pass: $P_t = Q + A^\top P_{t+1} A - A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A$
  - Compute gain: $K_t = (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A$
  - Compute action: $u_t = -K_t x_t$
- If $T \to \infty$
  - Solve P from **DARE** (discrete-time algebraic Riccati equation):

    $$P = Q + A^\top P A - A^\top P B (R + B^\top P B)^{-1} B^\top P A$$

  - Time-invariant gain: $K = (R + B^\top P B)^{-1} B^\top P A$
  - Policy: $u_t = -K x_t$
  - If $(A^\top, Q)$ and (A, B) are stabilizable: $u_t = -K x_t$ is a stable policy ($\rho(A - BK) < 1$)!

Stochastic Setting

$$\min_{x,u} \mathbb{E}_w \left[\sum_{t=1}^{T} \frac{1}{2} x_t^\top Q_t x_t + \frac{1}{2} u_t^\top R_t u_t \right]$$
$$\text{s.t.} \quad x_{t+1} = A_t x_t + B_t u_t + w_t, \forall t \in [1, T-1]$$

- w_t is zero-mean and i.i.d.: $\mathbb{E}[w_t] = 0$ and $\mathbb{E}[w_t w_t^\top] = W$
- It doesn't change the policy structure!
- Optimal cost-to-go (value) function: $V_t(x_t) = \frac{1}{2} x_t^\top P_t x_t + q_t$
- Step 1: $P_T = Q_T, q_T = 0$
- Step 2 (backward pass):
  - Backward pass: $P_t = Q_t + A_t^\top P_{t+1} A_t - A_t^\top P_{t+1} B_t (R_t + B_t^\top P_{t+1} B_t)^{-1} B_t^\top P_{t+1} A_t$
  - Backward pass: $q_t = q_{t+1} + \frac{1}{2} \mathrm{Tr}(W P_{t+1})$
  - Compute gain: $K_t = (R_t + B_t^\top P_{t+1} B_t)^{-1} B_t^\top P_{t+1} A_t$
  - Compute action: $u_t = -K_t x_t$
- Optimal cost: $\frac{1}{2} x_1^\top P_1 x_1 + q_1$

General LQR

$$\min_{x,u} \sum_{t=1}^{T} \frac{1}{2} \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top C_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top l_t$$

$$\text{s.t.} \quad x_{t+1} = F_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + g_t, \forall t \in [1, T-1]$$

- Notations:

$$C_t = \begin{bmatrix} Q_t & N_t \\ N_t^\top & R_t \end{bmatrix} \in \mathbb{R}^{(n+m)\times(n+m)} \succ 0$$

$$l_t = \begin{bmatrix} l_{x,t} \\ l_{u,t} \end{bmatrix} \in \mathbb{R}^{n+m}$$

$$F_t = [A_t \quad B_t] \in \mathbb{R}^{n\times(n+m)}$$

- Cost-to-go (value) functions: $V_t(x_t) = \frac{1}{2} x_t^\top P_t x_t + x_t^\top r_t + q_t$
- Verify it is true at time step T (last time step): Yes! $P_T = Q_T, r_T = l_{x,T}, q_T = 0$

General LQR

$$\min_{x,u} \sum_{t=1}^{T} \frac{1}{2} \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top C_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top l_t$$

$$\text{s.t.} \quad x_{t+1} = F_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + g_t, \forall t \in [1, T-1]$$

- Cost-to-go (value) functions: $V_t(x_t) = \frac{1}{2} x_t^\top P_t x_t + x_t^\top r_t + q_t$
- Bellman recursion:

$$V_t(x_t) = \min_u \underbrace{\frac{1}{2} x_t^\top Q_t x_t + \frac{1}{2} u^\top R_t u + u^\top N_t^\top x_t + x_t^\top l_{x,t} + u^\top l_{u,t}}_{\text{current step cost}} + \underbrace{V_{t+1}(A_t x_t + B_t u + g_t)}_{\text{cost-to-go}}$$

- Minimizer:
  - Define $R_t + B_t^\top P_{t+1} B_t = S_t$

$$u^* = -\underbrace{S_t^{-1}(B_t^\top P_{t+1} A_t + N_t^\top)}_{K_t} x_t - \underbrace{S_t^{-1}(l_{u,t} + B_t^\top P_{t+1} g_t + B_t^\top r_{t+1})}_{k_t}$$

General LQR

$$\min_{x,u} \sum_{t=1}^{T} \frac{1}{2} \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top C_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top l_t$$

$$\text{s.t.} \quad x_{t+1} = F_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + g_t, \forall t \in [1, T-1]$$

- Plug in u^* into $V_t(x_t)$:

$$P_t = Q_t + A_t^\top P_{t+1} A_t - K_t^\top S_t K_t$$

$$r_t = l_{x,t} + A_t^\top (P_{t+1} g_t + r_{t+1}) - K_t^\top S_t k_t$$

$$q_t = \frac{1}{2} g_t^\top P_{t+1} g_t + g_t^\top r_{t+1} + q_{t+1} - \frac{1}{2} k_t^\top S_t k_t$$

- Computing q_t is not necessary (needed if you care about the optimal cost $V_1(x_1)$)

General LQR Algorithm

$$\min_{x,u} \sum_{t=1}^{T} \frac{1}{2} \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top C_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top l_t$$

$$\text{s.t.} \quad x_{t+1} = F_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + g_t, \forall t \in [1, T-1]$$

- Initialize $P_T = Q_T, r_T = l_{x,T}$
- For $t = T-1$ to $t = 1$:
  - $S_t = R_t + B_t^\top P_{t+1} B_t$
  - $K_t = S_t^{-1}(B_t^\top P_{t+1} A_t + N_t^\top), k_t = S_t^{-1}(l_{u,t} + B_t^\top P_{t+1} g_t + B_t^\top r_{t+1})$
  - $P_t = Q_t + A_t^\top P_{t+1} A_t - K_t^\top S_t K_t$
  - $r_t = l_{x,t} + A_t^\top (P_{t+1} g_t + r_{t+1}) - K_t^\top S_t k_t$
  - $u_t = -K_t x_t - k_t$
- Similarly, return a time-variant *linear feedback controller*
- Similarly, solution will not change if there is zero-mean and i.i.d. noise
- Power of DP: complexity increases *cubically* as $\dim([x_t; u_t])$ increases, but *linearly* as T increases! But cannot easily handle state or action constraints.

What if the Problem is Nonconvex?

- When convex, relatively straightforward
  - In particular, quadratic programming (QP)
  - QP w/o state and action constraints: LQR
- When nonconvex:
  - Local optimization method: sequential quadratic/convex programming (SQP/SCP)
  - Dynamic programming (DP) + local optimization: iLQR/DDP
  - Sampling-based methods: MPPI and variants, CEM, CMA-ES

$$\min_{x,u} \sum_{t=1}^{T} c_t(x_t, u_t)$$

$$\text{s.t.} \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ x_t \in X_t \\ u_t \in U_t \end{cases}$$

SQP: Sequential Quadratic Programming

- Initialize a trajectory $x_{1:T}^{(1)}, u_{1:T}^{(1)}$
- At every iteration (i):
  - Affinize the dynamics around $x_{1:T}^{(i)}, u_{1:T}^{(i)}$
    - It is NOT linearization around some fixed equilibrium point

$$x_{t+1} \approx \nabla_{x_t,u_t} f_t(x_t^{(i)}, u_t^{(i)}) \begin{bmatrix} x_t - x_t^{(i)} \\ u_t - u_t^{(i)} \end{bmatrix} + f_t(x_t^{(i)}, u_t^{(i)})$$

  - Quadratic approximation of the cost around $x_{1:T}^{(i)}, u_{1:T}^{(i)}$

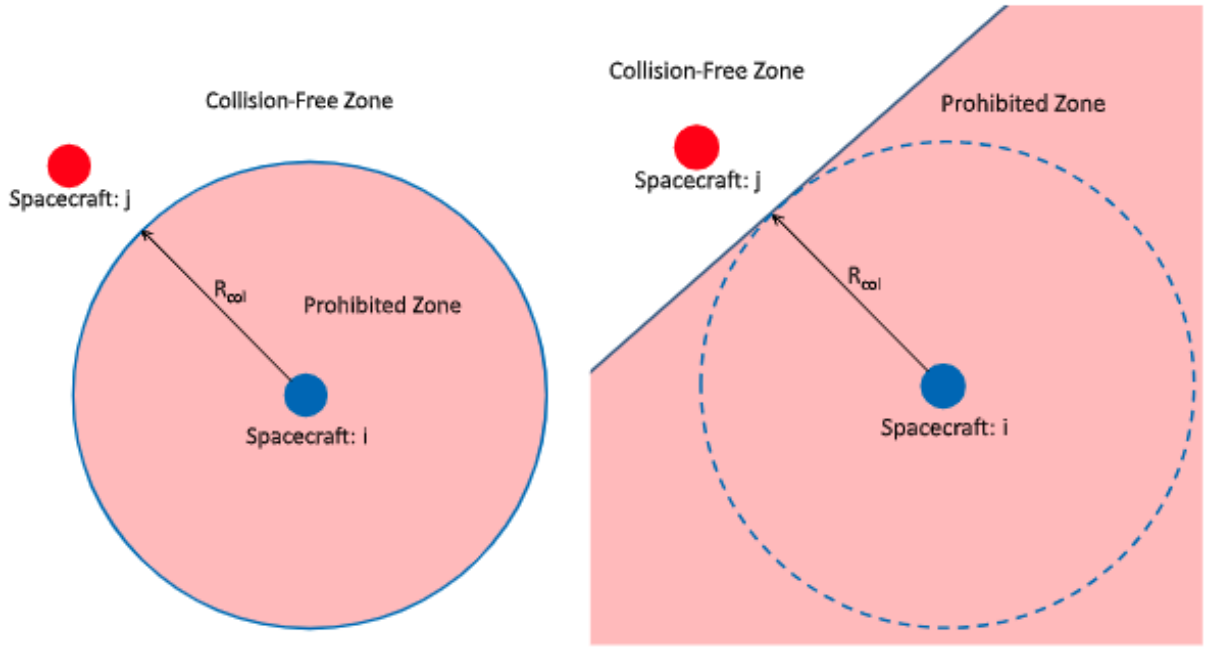
$$c_t(x_t, u_t) \approx c_t(x_t^{(i)}, u_t^{(i)}) + \nabla_{x_t,u_t} c_t(x_t^{(i)}, u_t^{(i)}) \begin{bmatrix} x_t - x_t^{(i)} \\ u_t - u_t^{(i)} \end{bmatrix}$$

$$+ \frac{1}{2} \begin{bmatrix} x_t - x_t^{(i)} \\ u_t - u_t^{(i)} \end{bmatrix}^\top \nabla^2_{x_t,u_t} c_t(x_t^{(i)}, u_t^{(i)}) \begin{bmatrix} x_t - x_t^{(i)} \\ u_t - u_t^{(i)} \end{bmatrix}$$

SQP: Sequential Quadratic Programming

- Initialize a trajectory $x_{1:T}^{(1)}, u_{1:T}^{(1)}$
- At every iteration (i):
  - Affinize the constraint around $x_{1:T}^{(i)}, u_{1:T}^{(i)}$

$$d_t^{\min,(i)} \leq D_t^{(i)} \begin{bmatrix} x_t \\ u_t \end{bmatrix} \leq d_t^{\max,(i)}$$



a) Nonconvex prohibited zone b) Convex approximation of prohibited zone

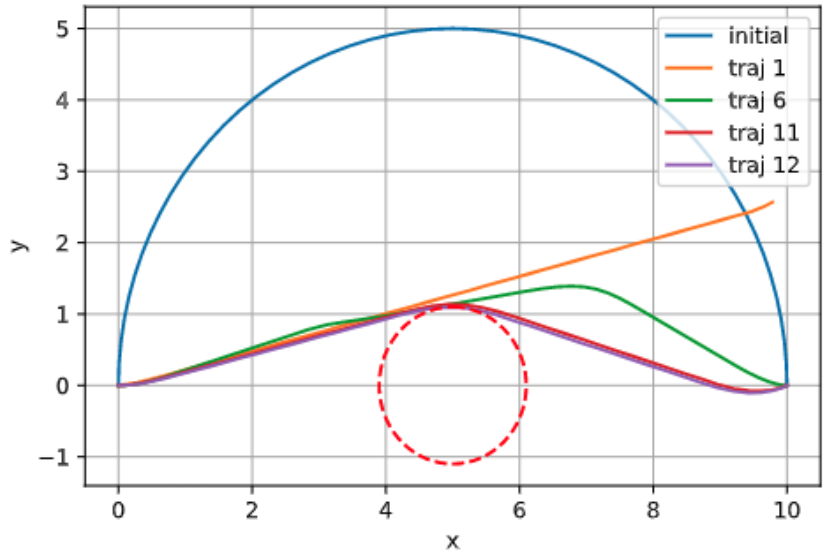
Fig. 2 Convexification of the two-dimensional (2-D) collision-avoidance constraint.

SQP: Sequential Quadratic Programming

- At every iteration (i):
  - Solve the following QP problem.
  - The last constraint is a *trust region* (not change too much from the last iteration)
  - The new solution is $x_{1:T}^{(i+1)}, u_{1:T}^{(i+1)}$
- Convergence criteria: $x_{1:T}^{(i+1)}, u_{1:T}^{(i+1)}$ and $x_{1:T}^{(i)}, u_{1:T}^{(i)}$ are close enough

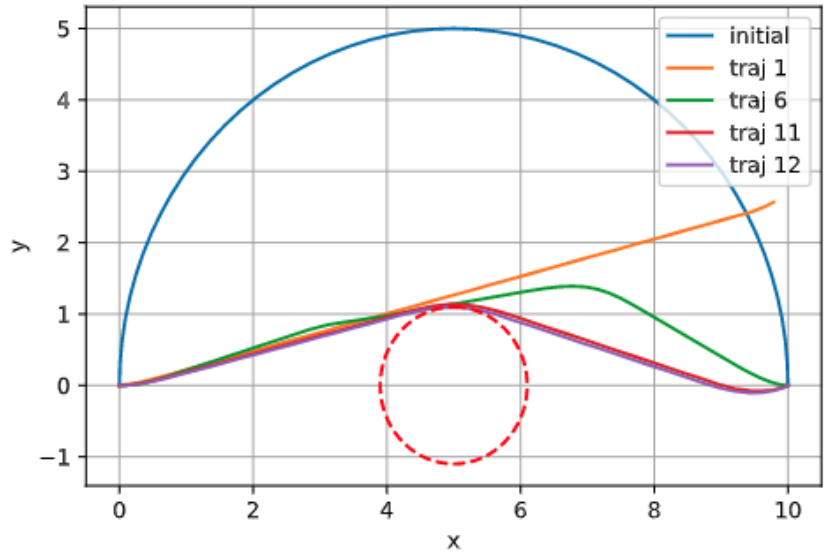
$$\min_{x,u} \sum_{t=1}^{T} \frac{1}{2} \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top C_t^{(i)} \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top l_t^{(i)}$$

$$\text{s. t.} \begin{cases} x_{t+1} = A_t^{(i)} x_t + B_t^{(i)} u_t + g_t^{(i)} \\ d_t^{\min,(i)} \le D_t^{(i)} \begin{bmatrix} x_t \\ u_t \end{bmatrix} \le d_t^{\max,(i)} \\ \left\| \begin{bmatrix} x_t \\ u_t \end{bmatrix} - \begin{bmatrix} x_t^{(i)} \\ u_t^{(i)} \end{bmatrix} \right\| \le \epsilon \end{cases}$$



SQP: Sequential Quadratic Programming

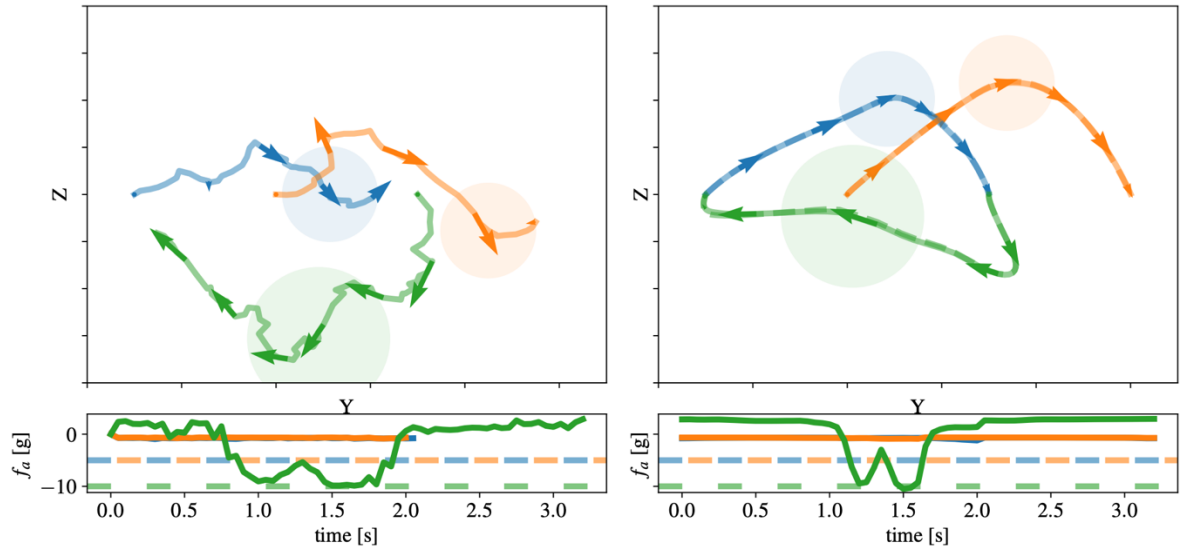
- How to find the initialization $x_{1:T}^{(1)}, u_{1:T}^{(1)}$?
  - In SQP-MPC, from the last time step's solution (shifted by one step)
  - Manually design (heuristic)
  - Search (e.g., RRT-style algorithms)
  - Learning



Neural-Swarm2: Planning and Control of Heterogeneous Multirotor Swarms Using Learned Interactions

Publisher: IEEE | Cite This | PDF

Guanya Shi ; Wolfgang Hönig ; Xichen Shi ; Yisong Yue ; Soon-Jo Chung All Authors



AO-RRT initialization + SCP

More General: Sequential Convex Programming

- More to read: https://web.stanford.edu/class/ee364b/lectures/seq_slides.pdf

we consider nonconvex problem

$$\begin{array}{ll} \text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq 0, \quad i = 1, \ldots, m \\ & h_i(x) = 0, \quad j = 1, \ldots, p \end{array}$$

with variable $x \in \mathbf{R}^n$

- f_0 and f_i (possibly) nonconvex
- h_i (possibly) non-affine

Basic idea of SCP

- maintain estimate of solution $x^{(k)}$, and convex **trust region** $\mathcal{T}^{(k)} \subset \mathbf{R}^n$
- form convex approximation $\hat{f}_i$ of f_i over trust region $\mathcal{T}^{(k)}$
- form affine approximation $\hat{h}_i$ of h_i over trust region $\mathcal{T}^{(k)}$
- $x^{(k+1)}$ is optimal point for approximate convex problem

$$\begin{array}{ll} \text{minimize} & \hat{f}_0(x) \\ \text{subject to} & \hat{f}_i(x) \leq 0, \quad i = 1, \ldots, m \\ & \hat{h}_i(x) = 0, \quad i = 1, \ldots, p \\ & x \in \mathcal{T}^{(k)} \end{array}$$

- Summary of SCP/SQP: a local optimization method for nonconvex problem
- It is a heuristic:
  - It can fail to find optimal (or even feasible) solution
  - Results highly depend on initialization
  - Often works very well in practice!

iLQR: Iterative Linear Quadratic Regulator

- iLQR also linearizes the dynamics and makes a quadratic approximation of costs around $x_{1:T}^{(i)}, u_{1:T}^{(i)}$
- But it leverages the structure of the Riccati equations (from LQR) to come up with a second-order update to the trajectory after a single *backward pass* of the Riccati equation followed by a *forward simulation*
- Therefore the convergence is fast and robust!
- Doesn't directly deal with constraints (will need penalty methods such as augmented Lagrangian)

$$\min_{x,u} \sum_{t=1}^{T} c_t(x_t, u_t)$$

$$\text{s.t.} \quad x_{t+1} = f_t(x_t, u_t)$$

iLQR: Iterative Linear Quadratic Regulator

- At every iteration (i):
  - LQR backward pass on the approximated quadratic problem:

$$u_t = -K_t^{(i)} x_t - k_t^{(i)}$$

  - Forward pass: roll out $u_t = -K_t^{(i)} x_t - k_t^{(i)}$ with the *real nonlinear dynamics*
  - It is why iLQR could be fast & robust (it leverages the structure of the Riccati equation with a *feedback* structure)
  - The new solution is $x_{1:T}^{(i+1)}, u_{1:T}^{(i+1)}$ from the forward pass
- Convergence criteria: $x_{1:T}^{(i+1)}, u_{1:T}^{(i+1)}$ and $x_{1:T}^{(i)}, u_{1:T}^{(i)}$ are close enough
- Often reformulate as $\delta x_t^{(i)} = x_t - x_t^{(i)}$

$$\min_{x,u} \sum_{t=1}^{T} \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top C_t^{(i)} \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top l_t^{(i)}$$

$$\text{s.t.} \quad x_{t+1} = A_t^{(i)} x_t + B_t^{(i)} u_t + g_t^{(i)}, \forall t \in [1, T-1]$$

Connection to Newton's Method

- Newton's method for minimizing a function $h(x)$: $\mathbb{R}^n \to \mathbb{R}$
  - Compute its gradient $g^\top = \nabla_x h(x_t)$
  - Compute its Hessian $H = \nabla_x^2 h(x_t)$
  - Newton's method solves:

$$x_{t+1} = \text{argmin}_x \frac{1}{2}(x - x_t)^\top H(x - x_t) + g^\top(x - x_t)$$

  - GD (gradient descent) can be viewed as a special case $H = I/\eta$ (η is the learning rate)

- iLQR is an approximation of Newton's method for solving

$$\min_u c_1(x_1, u_1) + c_2(f_1(x_1, u_1), u_2) + c_3(f_2(f_1(x_1, u_1), u_2), u_3) + \cdots$$

- To get a full second-order method, will need second-order approximation of f_t:
  - This is DDP (differential dynamic programming)!

$$x_{t+1} \approx f_t\left(x_t^{(i)}, u_t^{(i)}\right) + \nabla_{x_t,u_t} f_t\left(x_t^{(i)}, u_t^{(i)}\right)\begin{bmatrix}\delta x_t^{(i)} \\ \delta u_t^{(i)}\end{bmatrix} + \frac{1}{2}(\nabla^2_{x_t,u_t} f_t\left(x_t^{(i)}, u_t^{(i)}\right) \cdot \begin{bmatrix}\delta x_t^{(i)} \\ \delta u_t^{(i)}\end{bmatrix})\begin{bmatrix}\delta x_t^{(i)} \\ \delta u_t^{(i)}\end{bmatrix}$$

Differential Dynamic Programming

- iLQR is an approximation of Newton's method for solving

$$\min_{u} c_1(x_1, u_1) + c_2(f_1(x_1, u_1), u_2) + c_3(f_2(f_1(x_1, u_1), u_2), u_3) + \cdots$$

- To get a full second-order method, will need second-order approximation of f_t:
  - This is DDP (differential dynamic programming)!

$$x_{t+1} \approx f_t\left(x_t^{(i)}, u_t^{(i)}\right) + \nabla_{x_t,u_t} f_t\left(x_t^{(i)}, u_t^{(i)}\right) \begin{bmatrix} \delta x_t^{(i)} \\ \delta u_t^{(i)} \end{bmatrix} + \frac{1}{2}\left(\nabla^2_{x_t,u_t} f_t\left(x_t^{(i)}, u_t^{(i)}\right) \cdot \begin{bmatrix} \delta x_t^{(i)} \\ \delta u_t^{(i)} \end{bmatrix}\right) \begin{bmatrix} \delta x_t^{(i)} \\ \delta u_t^{(i)} \end{bmatrix}$$

- Scalar 2-step system example: $\min_{u_1} c_1(u_1) + c_2\big(x_2 = f_1(x_1, u_1)\big)$
  - $\nabla^2_{u_1} f_1$ will appear when compute $\nabla^2_{u_1} c_2$
- Summarization of iLQR/DDP:
  - Can be viewed as (approximated) Newton's methods for nonconvex problems
  - SQP without state and action constraints
  - Use dynamic programming: leverage Riccati equation
  - Compare to SQP: complexity increases *linearly* as horizon increases

Sampling-based TO/MPC

- General idea:
  - Sample N control action sequences from some distribution: $u_{1:T}^{(1)}, \cdots, u_{1:T}^{(N)}$
  - Roll out them on the nonlinear dynamics to evaluate their costs: $C^{(1)}, \cdots, C^{(N)}$
  - Compute weights based on costs: $w^{(1)}, \cdots, w^{(N)}$
  - Update the distribution using $u_{1:T}^{(1)}, \cdots, u_{1:T}^{(N)}$ and $w^{(1)}, \cdots, w^{(N)}$
  - Stop when convergence
- Example:
  - MPPI (model predictive path integral control) and variants
  - CEM (cross-entropy method)
  - CMA-ES (covariance matrix adaptation evolutionary strategy)

Sampling-based TO/MPC

- MPPI:
  - Sample N control sequences from $\mathcal{N}(\mu, \Sigma)$: $u_{1:T}^{(1)}, \cdots, u_{1:T}^{(N)}$
  - Roll out to evaluate costs: $C^{(1)}, \cdots, C^{(N)}$
  - Compute weights based on costs: $w^{(i)} = \frac{\exp(-\frac{1}{\lambda}C^{(i)})}{\sum_j \exp(-\frac{1}{\lambda}C^{(j)})}$
    - $\lambda > 0$ is the temperature
  - Update the distribution: $\mu = \sum_i w^{(i)} u_{1:T}^{(i)}$
  - Stop when convergence. Output μ.

$$\min_u C(u_{1:T}) = \sum_{t=1}^{T} c_t(x_t, u_t)$$

$$\text{s.t.} \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ u_t \in U_t \end{cases}$$

Sampling-based TO/MPC

- CEM:
  - Sample N control sequences from $\mathcal{N}(\mu, \Sigma)$: $u_{1:T}^{(1)}, \cdots, u_{1:T}^{(N)}$
  - Roll out to evaluate costs: $C^{(1)}, \cdots, C^{(N)}$
  - Select the K best samples ($K \ll N$): $u_{1:T}^{(1)}, \cdots, u_{1:T}^{(K)}$
  - Update the mean: $\mu = \sum_{i=1}^{K} \frac{1}{K} u_{1:T}^{(i)}$
  - Update the covariance: $\Sigma = \sum_{i=1}^{K} \frac{1}{K} (u_{1:T}^{(i)} - \mu)(u_{1:T}^{(i)} - \mu)^\top$
  - Stop when convergence. Output μ.

$$\min_{u} C(u_{1:T}) = \sum_{t=1}^{T} c_t(x_t, u_t)$$

$$\text{s.t.} \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ u_t \in U_t \end{cases}$$

Sampling-based TO/MPC

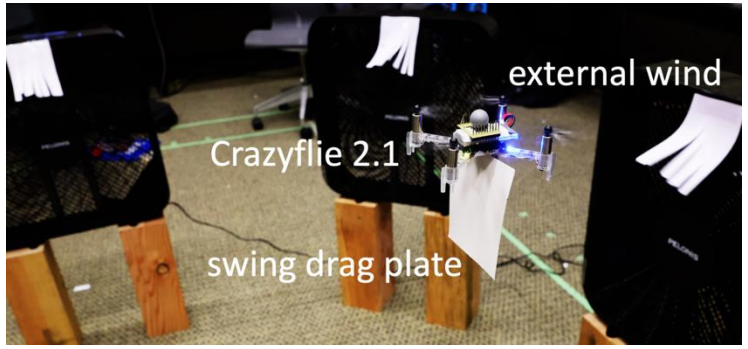
- A general framework (CMA-ES or DMD-MPC):
  - Sample N control sequences from $\mathcal{N}(\mu, \Sigma)$: $u_{1:T}^{(1)}, \cdots, u_{1:T}^{(N)}$
  - Roll out to evaluate costs: $C^{(1)}, \cdots, C^{(N)}$
  - Compute weights based on costs: $w^{(i)}$ (e.g., exponential, top-K, etc)
  - Update the mean: $\mu \leftarrow (1-\gamma_\mu)\mu + \gamma_\mu \sum_i w^{(i)} u_{1:T}^{(i)}$ (γ_μ is the learning rate)
  - Update the variance: $\Sigma \leftarrow (1-\gamma_\Sigma)\Sigma + \gamma_\Sigma \sum_i (u_{1:T}^{(i)} - \mu)(u_{1:T}^{(i)} - \mu)^\top$
  - Stop when convergence. Output μ.
- Even more general: use a parameterized distribution $\mathcal{N}(\theta)$ (not necessarily Gaussian)

$$\min_u C(u_{1:T}) = \sum_{t=1}^{T} c_t(x_t, u_t)$$

$$\text{s.t.} \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ u_t \in U_t \end{cases}$$

How to Make it Receding Horizon (MPC)?

- At time step $t+1$: reuse μ_t and Σ_t (shift one index)!
- Why this framework is more and more popular:
  - Easy to handle complex costs/dynamics (nondifferentiable, DNN, etc)
  - Massive parallelization on GPUs
  - Easy to implement
- 8192 samples, $H = 40$, can do ~80Hz on a laptop (GTX 1660 Ti)!



Information Theoretic MPC for Model-Based Reinforcement Learning

Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews,
James M. Rehg, Byron Boots, and Evangelos A. Theodorou

Deep Model Predictive Optimization

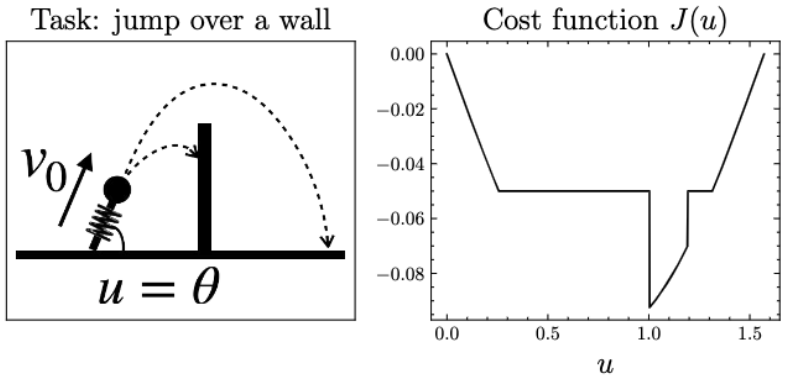
Jacob Sacks*, Rwik Rana*, Kevin Huang*, Alex Spitzer*, Guanya Shi†, and Byron Boots*

Algorithm 2: MPPI

Given: $\mathbf{F}$: Transition Model;
K: Number of samples;
T: Number of timesteps;
$(\mathbf{u}_0, \mathbf{u}_1, ...\mathbf{u}_{T-1})$: Initial control sequence;
Σ, ϕ, q, λ: Control hyper-parameters;
while *task not completed* **do**
  $\mathbf{x}_0 \leftarrow$ GetStateEstimate();
  for $k \leftarrow 0$ **to** $K-1$ **do**
    $\mathbf{x} \leftarrow \mathbf{x}_0$;
    Sample $\mathcal{E}^k = \{\epsilon_0^k, \epsilon_1^k, \dots \epsilon_{T-1}^k\}$;
    for $t \leftarrow 1$ **to** T **do**
      $\mathbf{x}_t \leftarrow \mathbf{F}(\mathbf{x}_{t-1}, \mathbf{u}_{t-1} + \epsilon_{t-1}^k)$;
      $S(\mathcal{E}^k) \mathrel{+}= \mathbf{q}(\mathbf{x}_t) + \lambda \mathbf{u}_{t-1}^{\mathrm{T}} \Sigma^{-1} \epsilon_{t-1}^k$;
    $S(\mathcal{E}^k) \mathrel{+}= \phi(\mathbf{x}_T)$;
  $\beta \leftarrow \min_k[S(\mathcal{E}^k)]$;
  $\eta \leftarrow \sum_{k=0}^{K-1} \exp\left(-\frac{1}{\lambda}(S(\mathcal{E}^k) - \beta)\right)$;
  for $k \leftarrow 0$ **to** $K-1$ **do**
    $w(\mathcal{E}^k) \leftarrow \frac{1}{\eta} \exp\left(-\frac{1}{\lambda}(S(\mathcal{E}^k) - \beta)\right)$;
  for $t \leftarrow 0$ **to** $T-1$ **do**
    $\mathbf{u}_t \mathrel{+}= \sum_{k=1}^{K} w(\mathcal{E}^k) \epsilon_t^k$;
  SendToActuators($\mathbf{u}_0$);
  for $t \leftarrow 1$ **to** $T-1$ **do**
    $\mathbf{u}_{t-1} \leftarrow \mathbf{u}_t$;
  $\mathbf{u}_{T-1} \leftarrow$ Intialize($\mathbf{u}_{T-1}$);

Some Insights about MPPI

- Main challenges of MPC: Hard to solve when problem is highly nonconvex or high-dim
  - Gradient-based methods struggle or fail (especially in discontinuous cases)
- Sampling-based MPC (e.g., MPPI) is an alternative
  - Limited agility and little theoretical understanding



Model Predictive Path Integral Control (MPPI)

1. Sample N control seqs from $\mathcal{N}(U, \Sigma)$: $U^{(1)}, \cdots, U^{(N)}$
2. Roll out to evaluate costs: $J^{(1)}, \cdots, J^{(N)}$
3. Compute weights:

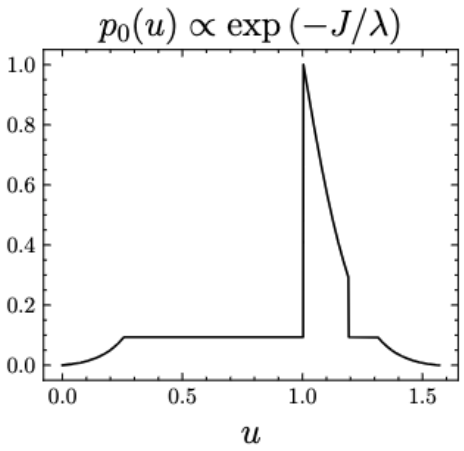
   $w^{(i)} = \frac{\exp(-\frac{1}{\lambda}J^{(i)})}{\sum_j \exp(-\frac{1}{\lambda}J^{(j)})}$. $\lambda > 0$ is the temperature
4. Output $U^+ = \sum_i w^{(i)} U^{(i)}$

Some Insights about MPPI

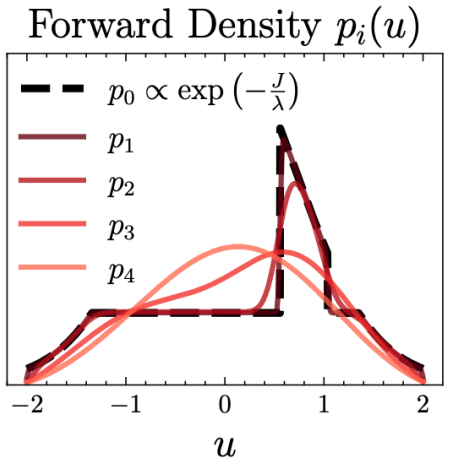
MPPI

1. Sample N control seqs from $\mathcal{N}(U, \Sigma)$: $U^{(1)}, \cdots, U^{(N)}$
2. Roll out to evaluate costs: $J^{(1)}, \cdots, J^{(N)}$
3. Compute weights: $w^{(i)} = \frac{\exp(-\frac{1}{\lambda}J^{(i)})}{\sum_j \exp(-\frac{1}{\lambda}J^{(j)})}$
4. Output $U^+ = \sum_i w^{(i)} U^{(i)}$

Target distribution p_0 (corresponding to J)



"Corrupted" distribution p_i
$p_\Sigma = p_0 * \mathcal{N}(0, \Sigma)$



Theorem (informal) [Pan* and Yi* et al., NeurIPS'24]

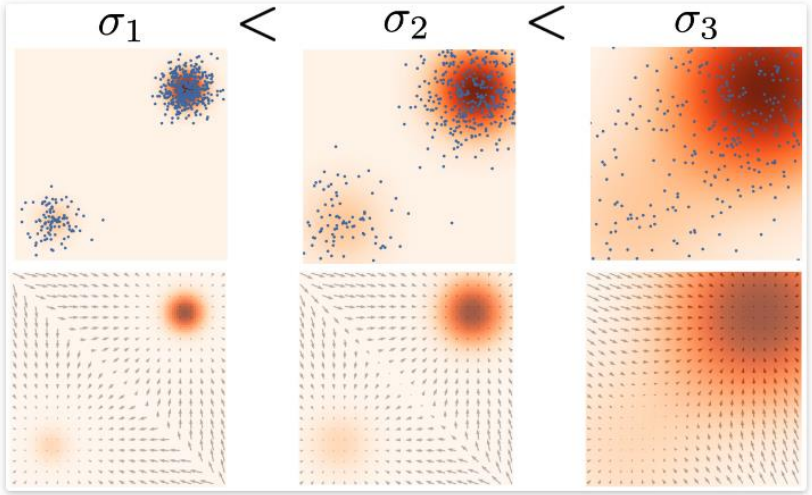
As $N \to \infty$, $U^+ = U + \Sigma \cdot \nabla \log p_\Sigma(U)$ where $p_\Sigma = p_0 * \mathcal{N}(0, \Sigma)$

- ❑ MPPI is a one-step ascent with a Σ-conditioned score function $\nabla \log p_\Sigma$!
  - It is (roughly) doing a "denoising" step
- ❑ Langevin dynamics in score-based diffusion model: $x_{t+1} = x_t + \frac{\alpha_i}{2} s_\theta(x_t, \sigma_i) + \sqrt{\alpha_i} z_t$

[Model-based Diffusion for Trajectory Optimization, Pan* and Yi* et al., NeurIPS'24]
[Full-Order Sampling-Based MPC for Torque-Level Locomotion Control via Diffusion-Style Annealing, Xue* and Pan* et al., under review]

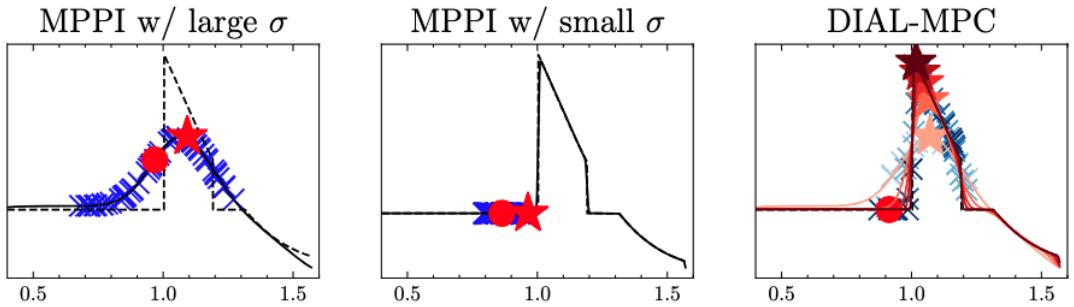
Diffusion-Inspired Annealing for Legged MPC

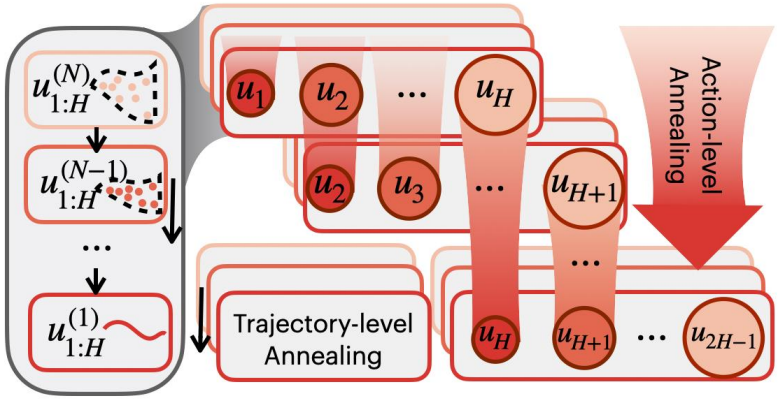
- Annealing in score-based diffusion: start from large noise and gradually anneal down



Generative Modeling by Estimating Gradients of the Data Distribution, Song and Ermon, NeurIPS'19

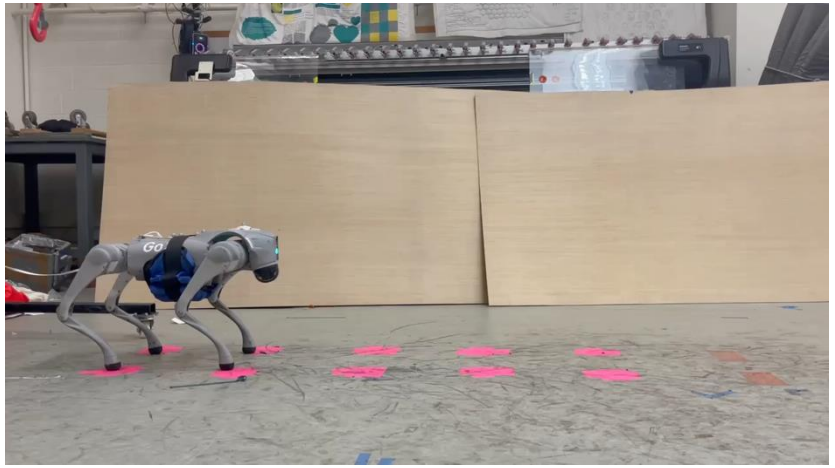
- DIAL-MPC (Diffusion-Inspired Annealing for Legged MPC)

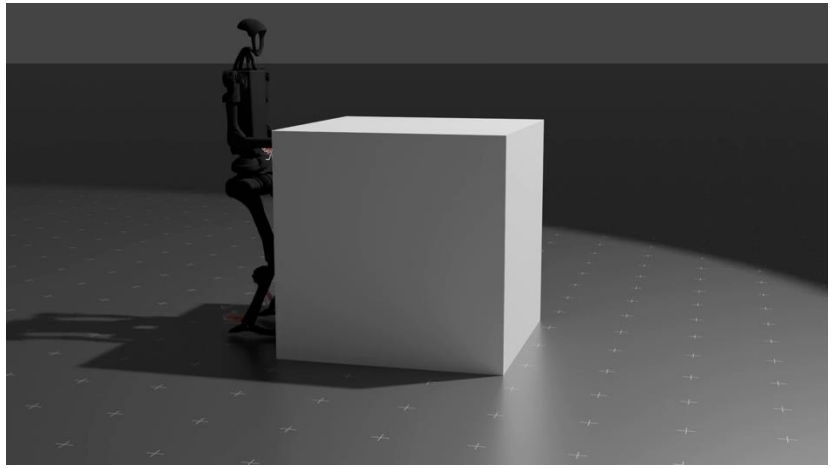




[Full-Order Sampling-Based MPC for Torque-Level Locomotion Control via Diffusion-Style Annealing, Xue* and Pan* et al., under review]

Diffusion-Inspired Annealing for Legged MPC





- ❑ DIAL-MPC enable 50Hz, torque-level, precise locomotion control using *full-order* dynamics
  - Directly roll out in parallel physics-based simulation
  - No reduced-order modeling, convexification, or predefined contact sequences
  - Training-free and plug-and-play
  - Zero-shot adaptation (e.g., payload)
  - Fully open-sourced
- ❑ Takeaway: Scaling *online search/optimization* is also powerful!
- ❑ More details: Chaoyi Pan's talk in Friday's locomotion seminar at 4pm in Wean 5415

[Full-Order Sampling-Based MPC for Torque-Level Locomotion Control via Diffusion-Style Annealing, Xue* and Pan* et al., under review]

Next Lecture: Modeling Learning

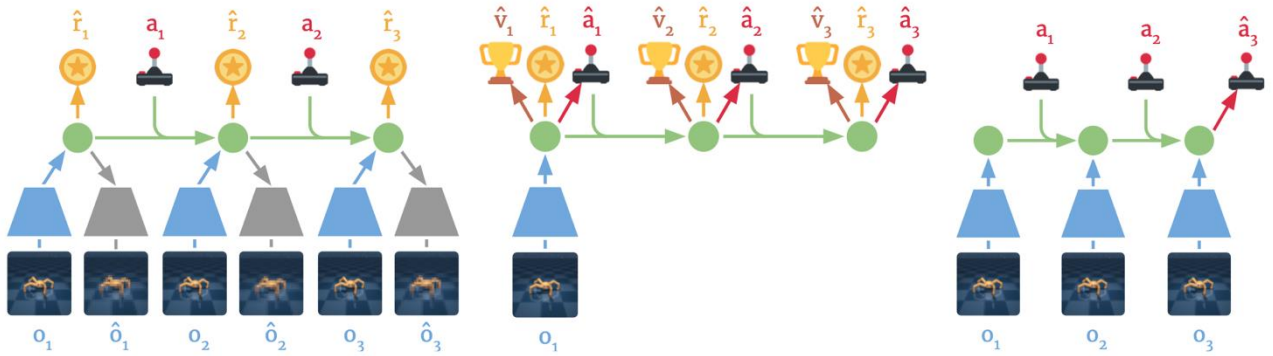
- Can we just use learning approaches to learn a model $f_\theta(x, u)$ and use it for control?
- Yes! But need to be very careful

$$\min_\theta \mathbb{E}_{x_t, u_t, x_{t+1} \sim \mathcal{D}} \| x_{t+1} - f_\theta(x_t, u_t) \|^2$$

- It is a *supervised regression* problem, but two fundamental differences:
  - Data generation is governed by the underlying (unknown) dynamics
  - ➢ I.e., we lose all good properties from i.i.d. data scheme
  - ➢ What policy we should use to generate data?

  - A good model doesn't necessarily mean good or easy control/planning
  - ➢ I.e., the regression error (learning objective) is NOT what we ultimately care about
  - ➢ What function class we should use to learn?

Next Lecture: Deep MBRL

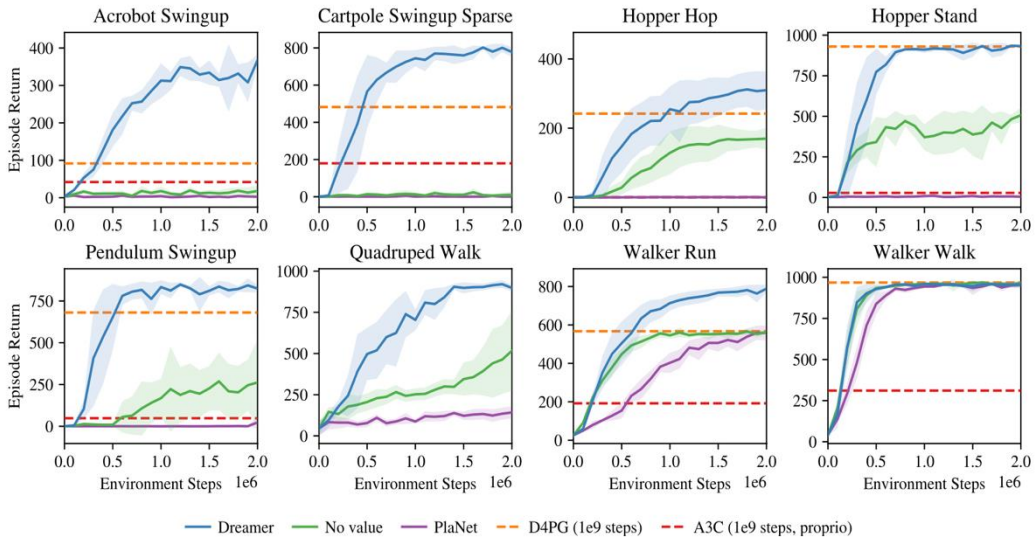


(a) Learn dynamics from experience (b) Learn behavior in imagination (c) Act in the environment

Representation model: $p(s_t \mid s_{t-1}, a_{t-1}, o_t)$

Transition model: $q(s_t \mid s_{t-1}, a_{t-1})$

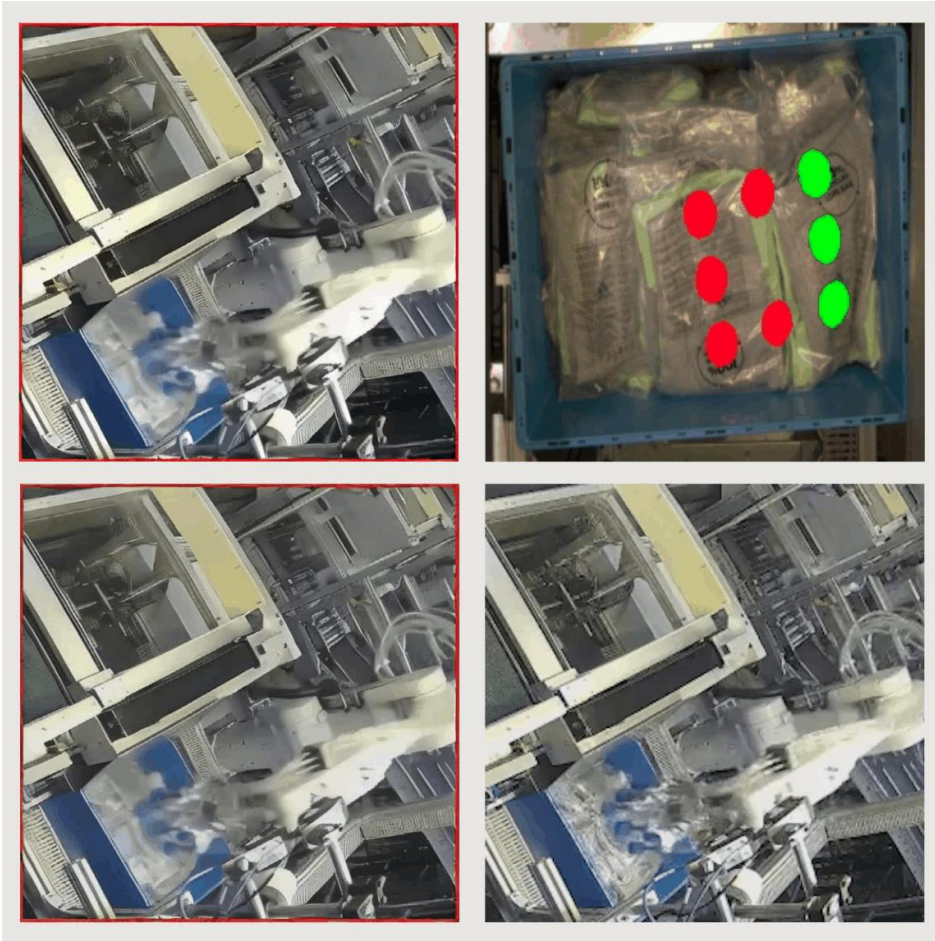
Reward model: $q(r_t \mid s_t)$.



Next Lecture: World Model and Robotics Foundation Model

RFM-1 simulates a picking action (bottom left) based on an initial image (top left) and prescribed item to pick (top right). The actual real-world pick result is on the bottom right.

covariant



Yann LeCun
@ylecun

Indeed, I do favor MPC over RL.
I've been making that point since at least 2016.
RL requires ridiculously large numbers of trials to learn any new task.
In contrast MPC is zero shot: If you have a good world model and a good task objective, MPC can solve new tasks without any task-specific learning.
That's the magic of planning.
It doesn't mean that RL is useless, but its use should be a last resort.

Thank You!